

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 01 -

INTRODUÇÃO À CONTAINERS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS	14

EMANIP

O QUE VEM POR AÍ?

Já parou pra pensar por que, às vezes, o código que funciona na sua máquina simplesmente não roda no servidor? Ou por que configurar um ambiente de desenvolvimento pode virar um verdadeiro pesadelo? Se você já passou por isso (ou teme passar), essa aula vai fazer muito sentido para você.

Nesta primeira etapa da nossa jornada, vamos entender o que são containers, por que eles revolucionaram a forma como desenvolvemos e distribuimos aplicações, e como o Docker se tornou uma ferramenta essencial na mochila de qualquer dev moderno. Vamos viajar no tempo das máquinas físicas até os dias de hoje, onde subir um ambiente completo pode ser tão simples quanto rodar um comando no terminal.

Você vai descobrir como containers resolvem problemas clássicos de compatibilidade, como eles se comparam com as máquinas virtuais e, claro, vai “botar a mão na massa” para rodar o seu primeiro container com Docker. É só o começo — e, acredite, essa caixinha mágica vai te acompanhar em muitos projetos daqui pra frente.

“Bora” começar?

HANDS ON

Agora que você já entendeu o contexto e a importância dos containers, chegou a hora de mergulhar de verdade nesse universo com a nossa sequência de videoaulas. Serão três vídeos práticos e diretos ao ponto, pensados para te ajudar a entender desde os conceitos básicos até a execução do seu primeiro container com Docker.

Durante as aulas, você verá explicações visuais, analogias simples e exemplos reais que mostrarão como os containers se encaixam no dia a dia do desenvolvimento. E claro: vai ter hands-on! Ou seja, você vai aprender não só com teoria, mas também com prática — comandos reais, execução de containers e tudo o que precisa para você dar os primeiros passos com segurança.

Dica: se quiser ir acompanhando, já prepara seu terminal e, se ainda não tiver, baixe o Docker [aqui](#).

Juntos, vamos instalar o Docker e rodar seu primeiro container com o famoso docker run hello-world. Também vamos te mostrar os principais comandos, aqueles que você vai usar no dia a dia:

```
docker ps
docker images
docker pull
docker run
docker stop
docker rm
dockerinspect
```

Pronto e pronta para o desafio? Então “bora” abrir o terminal e fazer acontecer!

Assim que você terminar de assistir os vídeos, você estará com o ambiente pronto para continuar nossa jornada nas próximas aulas, onde aprofundaremos o uso de imagens, Dockerfiles, volumes, redes e muito mais.

Vamos nessa? Dá o play no primeiro vídeo e “bora” aprender na prática!

SAIBA MAIS

Histórico da virtualização e evolução até os containers

Durante muito tempo, aplicações eram implantadas diretamente em servidores físicos. Cada servidor precisava ser configurado manualmente, com sistema operacional, dependências e todas as particularidades exigidas por aquela aplicação. Esse modelo trazia vários problemas: baixa flexibilidade, desperdício de recursos e alta complexidade para manutenção e escalabilidade.

A virtualização surgiu como resposta a esses desafios. Com ela, foi possível dividir um único servidor físico em várias máquinas virtuais (VMs), cada uma com seu próprio sistema operacional, drivers e bibliotecas. Isso melhorou o aproveitamento do hardware e permitiu que diferentes aplicações compartilhassem o mesmo servidor de forma isolada. No entanto, o custo computacional das VMs era alto. Cada instância virtual carregava um sistema operacional completo, ocupava bastante memória e levava mais tempo para inicializar.

Foi nesse contexto que os containers ganharam espaço. Em vez de virtualizar a máquina inteira, os containers compartilham o kernel do sistema operacional do host e isolam apenas o necessário: processos, rede, sistema de arquivos e recursos. Essa abordagem reduz drasticamente o uso de recursos, acelera a inicialização e facilita a portabilidade entre ambientes.

O avanço da tecnologia de containers foi possível graças a funcionalidades do kernel Linux como namespaces (que isolam recursos) e cgroups (que controlam o uso de CPU, memória e I/O). Com essa base técnica, ferramentas como o Docker surgiram para facilitar a criação, empacotamento e execução de containers de forma acessível e padronizada.

A transição da virtualização tradicional para o modelo baseado em containers reflete uma mudança no modo como desenvolvedores e times de infraestrutura pensam sobre entrega de software: menos foco na máquina, mais foco na aplicação. O resultado é um processo de deploy mais ágil, confiável e alinhado com práticas modernas de integração e entrega contínua.

Problemas de deploy tradicionais

Antes de containers e outras ferramentas modernas, o processo de colocar uma aplicação no ar costumava ser lento, frágil e cheio de incertezas. Cada ambiente era diferente — e isso gerava um dos erros mais clássicos: “na minha máquina funciona”.

O principal problema vinha da forma como os ambientes eram configurados. Devs criavam a aplicação num sistema com determinada versão de linguagem, dependências específicas, configurações locais, e depois tentavam replicar tudo isso em servidores que nem sempre estavam iguais. Às vezes, o servidor tinha outro sistema operacional, permissões diferentes ou até versões conflitantes de bibliotecas. O resultado? Falhas difíceis de rastrear e tempo perdido tentando “ajeitar o ambiente”.

Outro ponto crítico era a instalação manual. Era comum rodar scripts diretamente no servidor, instalar pacotes manualmente, criar diretórios e ajustar permissões na mão. Isso tornava o processo lento e difícil de reproduzir — e qualquer mudança pequena podia quebrar tudo. Isso sem contar que a documentação geralmente não acompanhava essas alterações.

Além disso, escalar uma aplicação não era simples. Rodar múltiplas instâncias da mesma aplicação exigia copiar e configurar tudo de novo em outra máquina, ou criar uma imagem de VM inteira pra subir em outro host — o que demandava tempo, memória e bastante coordenação.

Esse cenário também dificultava o rollback. Se uma nova versão dava problema em produção, não era fácil voltar para a anterior. Muitas vezes, o time precisava reconfigurar tudo de novo ou torcer para ter um backup funcional.

Esses desafios foram, aos poucos, mostrando a necessidade de soluções mais confiáveis, rápidas de configurar e fáceis de replicar. E é justamente aí que os containers entram: para isolar o ambiente, garantir consistência e simplificar todo esse processo de deploy, sem depender de tantas variáveis externas.

Conceito e definição de container

O conceito de container pode parecer abstrato no começo, especialmente se você nunca ouviu falar antes. Então, vamos simplificar:

Pense em um container como uma espécie de caixinha isolada onde você coloca uma aplicação e tudo que ela precisa para rodar: bibliotecas, dependências, arquivos de configuração e recursos específicos. Essa "caixinha" fica fechada, isolada do ambiente externo, garantindo que a aplicação dentro dela sempre rode da mesma forma, não importa onde ela esteja.

Tecnicamente falando, um container é um ambiente isolado no nível do sistema operacional. Diferente das máquinas virtuais (que simulam um computador completo, com sistema operacional e tudo), os containers compartilham o kernel (núcleo) do sistema operacional do host. Isso significa que containers não precisam carregar um sistema inteiro para funcionar — eles reutilizam o que já existe no host, reduzindo drasticamente o uso de recursos.

Dois mecanismos principais do kernel Linux tornam isso possível:

- **Namespaces:** são responsáveis por criar ambientes isolados. Por exemplo, cada container tem seu próprio espaço para processos, rede, sistema de arquivos e usuários. Isso garante que um container não interfira no outro e que aplicações possam rodar lado a lado, sem conflitos;
- **Cgroups (ControlGroups):** são utilizados pra controlar a quantidade de recursos (memória, CPU, disco) que cada container pode utilizar. Assim, mesmo que um container tente consumir mais recursos que o previsto, ele fica limitado ao que foi definido, garantindo estabilidade ao sistema inteiro.

Mas por que tudo isso importa?

Porque essa combinação permite que você tenha um ambiente padronizado, previsível e extremamente portátil. Uma aplicação empacotada em um container roda exatamente do mesmo jeito na sua máquina, num servidor de teste, em produção ou até mesmo na nuvem. Isso elimina aquele velho problema de “na minha máquina funciona”.

Para entender na prática:

Se você desenvolve uma aplicação web que usa Node.js em uma versão específica, antes você precisava instalar essa versão exata na máquina que rodaria sua aplicação em produção. Com containers, você simplesmente cria uma imagem que já contém exatamente essa versão do Node, as dependências e os arquivos

necessários. Ao subir essa imagem num servidor (com Docker, por exemplo), você garante que tudo rode exatamente como você planejou — sem erros surpresa.

Namespaces e cgroups: como containers funcionam de verdade?

Quando falamos que os containers são ambientes isolados que compartilham o kernel do sistema operacional, surge uma pergunta importante: como isso acontece, exatamente?

Dois mecanismos internos do kernel Linux são os responsáveis diretos por garantir esse isolamento e a distribuição controlada dos recursos. Eles se chamam Namespaces e cgroups.

Namespaces: separando ambientes

Pense em namespaces como paredes que separam diferentes ambientes no seu sistema operacional. Cada namespace isola certos aspectos do sistema, criando um espaço seguro e isolado para cada aplicação rodar. Assim, processos dentro de um namespace não têm acesso direto ao que acontece em outro.

No Linux, existem diferentes tipos de namespaces:

- **PID Namespace (Process ID):** cada container tem sua própria lista de processos. O processo número 1 em um container pode ser diferente do processo número 1 em outro container ou no host principal;
- **Network Namespace:** isola a rede, permitindo que cada container tenha seu próprio conjunto de interfaces de rede e tabelas de roteamento. Assim, containers diferentes podem usar portas idênticas sem conflitos;
- **Mount Namespace:** isola o sistema de arquivos. Cada container tem sua visão única dos arquivos e diretórios, sem enxergar os arquivos dos outros containers ou do host, a menos que explicitamente compartilhados;
- **UTS Namespace:** permite que cada container tenha seu próprio nome de host (hostname), independente dos outros;
- **IPC Namespace (Inter-Process Communication):** garante que mecanismos de comunicação interna entre processos (como filas, semáforos, memória compartilhada) estejam isolados por container;

- **UserNameSpace:** separa a identificação de usuários e grupos, permitindo que um usuário com privilégios dentro do container não tenha os mesmos privilégios no sistema operacional principal.

Esses namespaces são justamente a base para garantir que cada aplicação rode como se estivesse completamente isolada, sem nem perceber que outros containers existem por perto.

Cgroups: controlando os recursos

Agora que você entendeu como namespaces separam as aplicações, a pergunta que surge é: quem limita os recursos que cada container pode consumir? É aí que entram os cgroups.

Cgroups (abreviação para "controlgroups") são mecanismos do kernel Linux que permitem gerenciar e limitar recursos do sistema como CPU, memória, I/O e largura de banda de rede.

Imagine que você tenha vários containers rodando em um servidor. Se um deles começar a consumir muita memória ou processador, ele pode prejudicar a performance de todo o sistema. Com os cgroups, você pode definir limites claros:

- **CPU:** você define quanto cada container pode usar da CPU, evitando que um container sobrecarregue todo o sistema;
- **Memória:** permite definir limites rígidos de memória para cada container, evitando que um container tome toda a RAM disponível;
- **I/O (entrada/saída):** você pode definir prioridades e limites para o acesso ao disco ou a operações de leitura/escrita, garantindo estabilidade no servidor.

Cgroups, portanto, garantem que mesmo se um container apresentar problemas de desempenho ou bugs, o impacto sobre outros containers será mínimo ou inexistente.

Juntos, Namespaces e cgroups são as tecnologias-chave que fazem containers funcionarem. Com eles, aplicações são empacotadas em ambientes seguros, eficientes e isolados, permitindo que pessoas desenvolvedoras se concentrem mais no software e menos na infraestrutura.

Se você dominar esses conceitos, estará preparado(a) não apenas para usar containers com segurança, mas também para entender melhor o comportamento das suas aplicações em ambientes reais de produção.

Comandos básicos do Docker: guia prático para consultar e lembrar

Uma das melhores maneiras de fixar o que você aprendeu é colocar a “mão na massa”. Para isso, nada melhor do que ter uma lista rápida de consulta sempre que precisar. A seguir, temos os comandos mais importantes que você viu até agora, explicados de forma simples para você consultar, lembrar e usar no dia a dia.

Containers em execução e gerenciamento básico

Esses comandos ajudam a controlar e acompanhar seus containers. São os comandos que você mais vai usar diariamente:

- **Listar containers em execução:** `docker ps`;
- **Listar todos os containers (inclusive parados):** `docker ps -a`;
- **Executar um container a partir de uma imagem:** `docker run nome-da-imagem`. Exemplo: `docker run hello-world`;
- **Executar um container interativo (com acesso ao terminal):** `docker run -it nome-da-imagem /bin/bash`. Exemplo: `docker run -it ubuntu bash`;
- **Parar um container em execução:** `docker stop id-ou-nome-do-container`;
- **Iniciar novamente um container parado:** `docker start id-ou-nome-do-container`;
- **Remover um container parado:** `docker rm id-ou-nome-do-container`;
- **Forçar a remoção de um container (mesmo que ainda esteja rodando):** `docker rm -f id-ou-nome-do-container`.

Trabalhando com imagens

Imagens são as bases dos containers, então você precisa gerenciá-las bem. Esses comandos vão te ajudar a baixar, listar e remover imagens locais:

- **Listar imagens disponíveis localmente:** `docker images`;

- **Baixar uma imagem específica do Docker Hub:** `docker pull nome-da-imagem`. Exemplo: `docker pull node:20-alpine`;
- **Remover uma imagem localmente armazenada:** `docker rmi nome-ou-id-da-imagem`.

Inspecionando detalhes de containers

Inspecionar é importante para entender o que acontece dentro dos seus containers e diagnosticar possíveis problemas:

- **Visualizar logs de execução do container:** `docker logs id-ou-nome-do-container`;
- **Obter informações detalhadas sobre um container ou imagem:** `docker inspect id-ou-nome-do-container`.

Outros comandos úteis no dia a dia

Esses comandos facilitam muito a vida quando você precisa renomear containers ou executar comandos dentro deles rapidamente:

- **Renomear um container:** `docker rename nome-antigo nome-novo`;
- **Executar comandos em containers já em execução:** `docker exec -it nome-do-container comando`. Exemplo: `docker exec -it meu-container bash`;
- **Copiar arquivos do host para o container:** `docker cp arquivo host:/caminho-container`;
- **Copiar arquivos do container para o host:** `docker cp container:/caminho-arquivo destino-host`.

Limpendo o ambiente Docker

Manter o ambiente limpo é fundamental para evitar problemas com espaço em disco e garantir que tudo esteja funcionando bem:

- **Remover containers parados automaticamente:** `docker container prune`;
- **Remover imagens não utilizadas:** `docker image prune`;
- **Limpar todos os recursos Docker não utilizados:** `docker system prune`.

Com esses comandos na mão, você já está preparado(a) para lidar com Docker no seu dia a dia com muito mais agilidade e segurança. Consulte sempre que precisar relembrar ou revisar algo, porque dominar esses comandos faz toda a diferença no seu trabalho com containers.

EMANIP

O QUE VOCÊ VIU NESTA AULA?

Ao longo dessa aula, você deu os primeiros passos no universo dos containers e entendeu como eles se tornaram tão importantes para o desenvolvimento e deploy modernos.

Começamos explorando os desafios do passado: ambientes inconsistentes, dependências conflitantes e o alto custo de rodar múltiplas aplicações em máquinas virtuais. Em seguida, você conheceu os containers como solução para esses problemas — oferecendo isolamento leve, portabilidade e eficiência.

Parabéns por chegar até aqui! Agora sim, você tem uma base sólida pra construir, testar e escalar suas aplicações com containers.

REFERÊNCIAS

DOCKER INC. **Docker Documentation**. 2025. Disponível em: <https://docs.docker.com/get-started/>. Acesso em: 26 mai. 2025.

HIGHTOWER, K.; BURNS, B.; BEDA, J. **Kubernetes: Up and Running – Dive into the Future of Infrastructure**. 2. ed. Sebastopol: O'Reilly Media, 2019.

MERKEL, D. Docker: **lightweight Linux containers for consistent development and deployment**. **Linux Journal**, 2014. Disponível em: <https://dl.acm.org/doi/10.5555/2600239.2600241>. Acesso em: 26 mai. 2025.

PALAVRAS-CHAVE

Containers. Virtualização. Docker.

EMPRE



POSTECH